

IACV

Plano de Producao

Estrategia de ambientes, releases e rollback

IA Cloud Vision - NVR com Visao Computacional

Este documento descreve a estrategia completa para colocar e manter o IACV em producao com seguranca, separando ambientes, automatizando deploys, garantindo rollback rapido e reduzindo o impacto sobre os tenants em cada atualizacao.

Aplicacao classificada como missao critica.

Janela de manutencao planejada: 10-30 segundos por tenant.

MTTR alvo (rollback simples): < 5 minutos.

INFORMACOES DO DOCUMENTO

Projeto:	IA Cloud Vision (IACV)
Repositorio:	github.com/tarcisioazevedo/iacloud-vision
Registro de imagens:	ghcr.io/tarcisioazevedo/iacloud-vision
Stack:	Docker Swarm + Traefik + GHCR + GitHub Actions
Banco:	SQLite (Peewee ORM, migrations automaticas)
Versao do plano:	1.0
Data:	27/04/2026

Sumario

1.	Sumario Executivo	4
2.	Principios de Missao Critica	5
3.	Arquitetura de Ambientes	6
4.	Estrategia de Branches Git	8
5.	Pipeline CI/CD	10
6.	Estrategia de Testes	12
7.	Procedimento de Deploy	14
8.	Procedimento de Rollback	16
9.	Procedimento de Hotfix	18
10.	Migrations de Banco	20
11.	Monitoramento e Alertas	21
12.	Backup e Disaster Recovery	22
13.	Plano de Implementacao (90 dias)	23
14.	Runbooks Detalhados	24
Anexo A.	Comandos uteis	33
Anexo B.	Template de Postmortem	34

1. Sumario Executivo

O **IA Cloud Vision (IACV)** e um sistema NVR multi-tenant com visao computacional baseado em Frigate, com extensoes para licenciamento por portal, sincronizacao S3 de gravacoes e tema glassmorphism no frontend React/Vite. Cada tenant roda como um stack Docker Swarm independente, isolado por DNS via Traefik.

Este plano organiza o ciclo de vida da aplicacao em producao com tres pilares:

- 1. Ambientes separados** (DEV, CI, STAGING, PROD), com promocao explicita entre eles. Nada chega em PROD sem passar por STAGING. Nada vai pra STAGING sem passar pelo CI.
- 2. Pipeline automatizado** (GitHub Actions + GHCR), com build multi-arquitetura, testes obrigatorios e gate humano antes de PROD. A imagem que vai pra producao e *exatamente* a mesma testada em STAGING.
- 3. Deploy gradual + rollback rapido**, usando canario em 1 tenant interno, batch em 10% e so depois 100%, com `--update-failure-action rollback` do Docker Swarm para reverter automaticamente em falha de healthcheck

Meta de impacto: downtime por tenant entre 10 e 30 segundos por release. Para o usuario final isso e perceptivel apenas como uma reconexao do navegador. Recordings nao sao perdidos (continuam fluindo do go2rtc para o S3 mesmo durante o restart).

Por que isso importa: IACV e missao critica. Tenants pagam pela disponibilidade do video em tempo real e pelo historico de gravacoes. Cada minuto fora do ar e: (1) risco de seguranca para o cliente final, (2) perda de evidencia legal, (3) churn. O custo de uma janela de manutencao mal feita e ordens de grandeza maior que o custo de fazer o processo direito.

2. Principios de Missao Critica

Cinco principios guiam todas as decisoes deste plano. Quando houver duvida em alguma situacao, volte aqui.

Imutabilidade

A imagem que vai pra producao e a MESMA testada em staging. Nunca rebuildar para producao. Tag versionada (vX.Y.Z), nao :latest. Isso elimina toda classe de bugs por divergencia de ambiente.

Reversibilidade

Toda mudanca deve ter caminho de volta em < 5 minutos. Migrations destrutivas (DROP, RENAME) sao proibidas em release normal - exigem migration em duas fases (expand-and-contract).

Observabilidade

Nada e deployado sem ser observavel. Cada release adiciona metricas/logs do que mudou. Se nao da pra ver se quebrou, nao da pra deployar.

Gradualidade

1 tenant interno -> 10% -> 100%. Nunca all-at-once. O blast radius de qualquer bug fica contido nos primeiros minutos.

Automacao do tedioso, manualidade do critico

Build, test, push de imagem: tudo automatico. Promocao para producao: clique humano com aprovacao. Maquinas nao tem responsabilidade pelo que envolve risco.

Regra de bolso: se voce esta cansado, com sono, ou com pressa - **nao deploye**. Adie. Aplicacao critica e dia de semana, horario comercial, alguem disponivel para apagar incendio.

3. Arquitetura de Ambientes

Quatro ambientes com responsabilidades claras. Cada um e isolado dos demais (host, dados, credenciais).

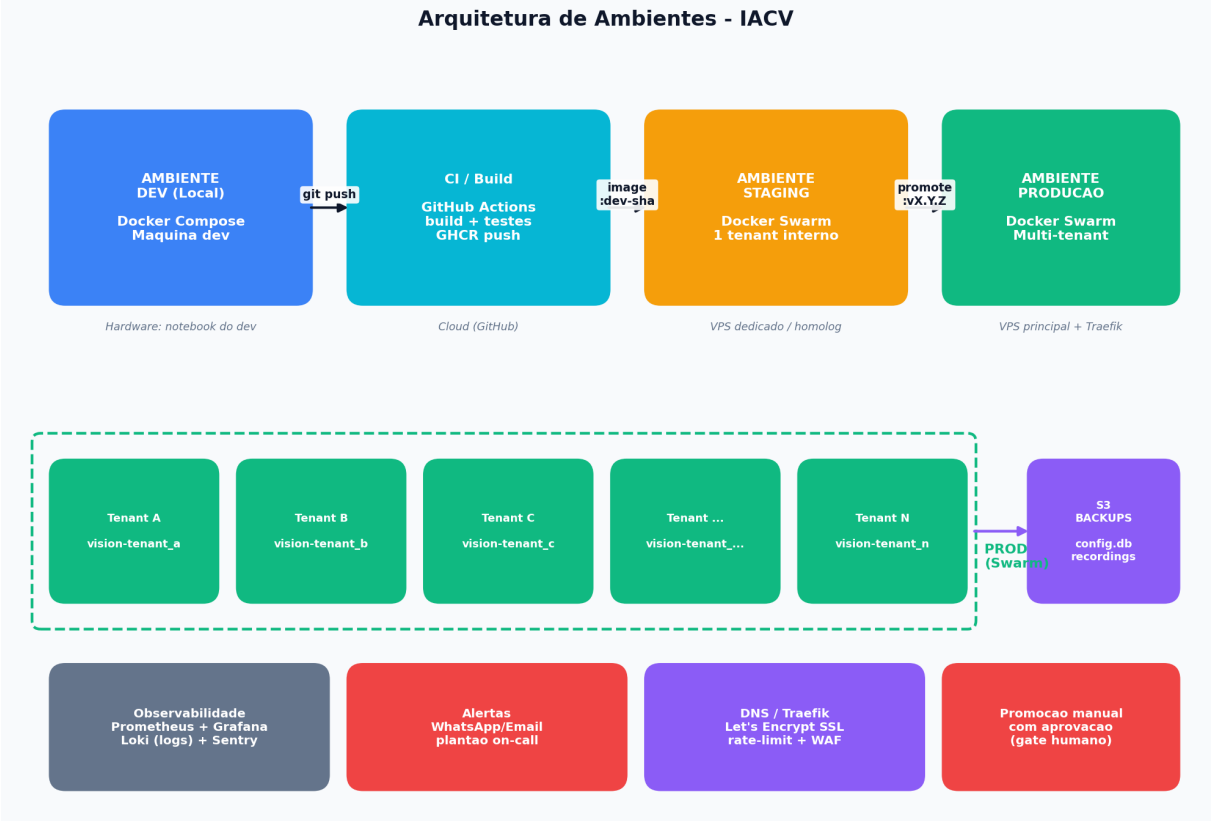


Figura 1 - Arquitetura completa de ambientes do IACV

3.1 Detalhamento dos ambientes

Ambiente	Onde roda	Dados	Acesso	Trafego
DEV (local)	Notebook do dev	Sinteticos / mock	Dev individual	Zero (interno)
CI	GitHub Actions	Sem dados	Pull requests	Zero (build apenas)
STAGING	VPS dedicado	Anonimizados	Equipe + QA	Tenant interno de teste
PROD	VPS principal + Swarm	Reais Multi-tenant	On-call + admins	Todos os tenants

3.2 Promocao entre ambientes

A promocao de uma versao da imagem entre ambientes **nao envolve rebuild** - so re-tagueamento e re-deploy:

```
# 1. CI builda e tagueia:
ghcr.io/.../iaccloud-vision:dev-{sha} # gerada por push em dev
ghcr.io/.../iaccloud-vision:vX.Y.Z # gerada por tag git
```

```
# 2. STAGING usa :dev-{sha} ou :vX.Y.Z-rcN
# 3. PROD usa apenas :vX.Y.Z (sem -rc, sem dev)

# Promocao = atualizar a tag no compose.cloud.yml e re-deploy
# A imagem em si nao muda - so o ponteiro.
```

3.3 Custos e dimensionamento

Componente	Especificacao	Custo mensal estimado
VPS Producao	8 vCPU, 16 GB, 500 GB SSD	R\$ 350
VPS Staging	4 vCPU, 8 GB, 100 GB SSD	R\$ 140
S3 Backups + Recordings	1 TB ativos + 5 TB archive	R\$ 250
GitHub Actions	5000 min/mes (free tier cobre)	R\$ 0
GHCR	Gratuito (publico), \$0.25/GB egress (privado)	R\$ 50
Total estimado		R\$ 790/mes

Nota: valores sao estimativas conservadoras para Brasil 2026 (Hetzner, AWS Sao Paulo, Backblaze B2). Ajustar conforme volume real de tenants e recordings.

4. Estrategia de Branches Git

Modelo simplificado de Git Flow, adaptado para releases frequentes. Cinco tipos de branch, cada um com regra clara.

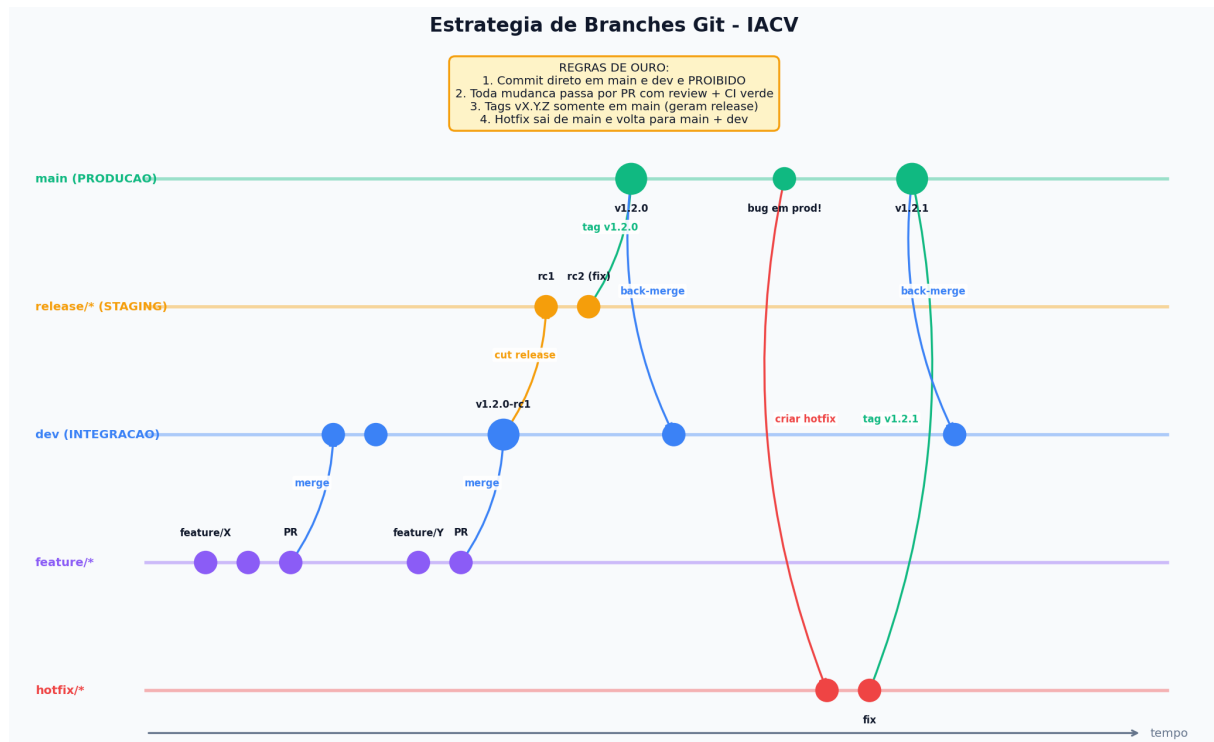


Figura 2 - Linha do tempo de branches: feature, dev, release, main, hotfix

4.1 Tipos de branch

Branch	Origem	Destino	Quando criar	Protecao
main	-	tags vX.Y.Z	Sempre existe	Sim (PR + 2 reviews)
dev	-	release/* / merge	Sempre existe	Sim (PR + 1 review)
feature/*	dev	dev (via PR)	Nova funcionalidade	Nao
release/vX.Y.Z	dev	main (via PR)	Cut de release	Sim (PR)
hotfix/*	main	main + dev	Bug critico em prod	Sim (PR + 2 reviews)

4.2 Configuracao de protecao no GitHub

Aplicar em **Settings > Branches > Add rule**:

Rule para 'main'

- Require a pull request before merging
- Require approvals: **2 aprovadores**
- Dismiss stale pull request approvals when new commits are pushed

- Require status checks to pass before merging: **ci.yml**
- Require branches to be up to date before merging
- Require linear history (no merge commits)
- Do not allow bypassing the above settings (mesmo para admin)
- Restrict who can push (apenas mantenedores via PR)

Rule para 'dev'

- Require a pull request before merging
- Require approvals: **1 aprovador**
- Require status checks to pass: **ci.yml**

4.3 Versionamento

Seguir **SemVer** (semver.org): vMAJOR.MINOR.PATCH

- **MAJOR (vX.0.0)**: Mudanca que quebra compatibilidade. Ex: API que muda contrato. Raro.
- **MINOR (v0.X.0)**: Funcionalidade nova compativel. Ex: novo tipo de detector.
- **PATCH (v0.0.X)**: Bugfix sem nova funcionalidade. Hotfixes sempre incrementam patch

Regra forte: commits diretos em main ou dev sao PROIBIDOS, mesmo para o admin. Toda mudanca passa por PR. Isso forza review, CI, e cria rastro de auditoria.

5. Pipeline CI/CD

O projeto ja tem 3 workflows funcionais. Este capitulo mostra como eles se conectam e quais ajustes implementar para missao critica.

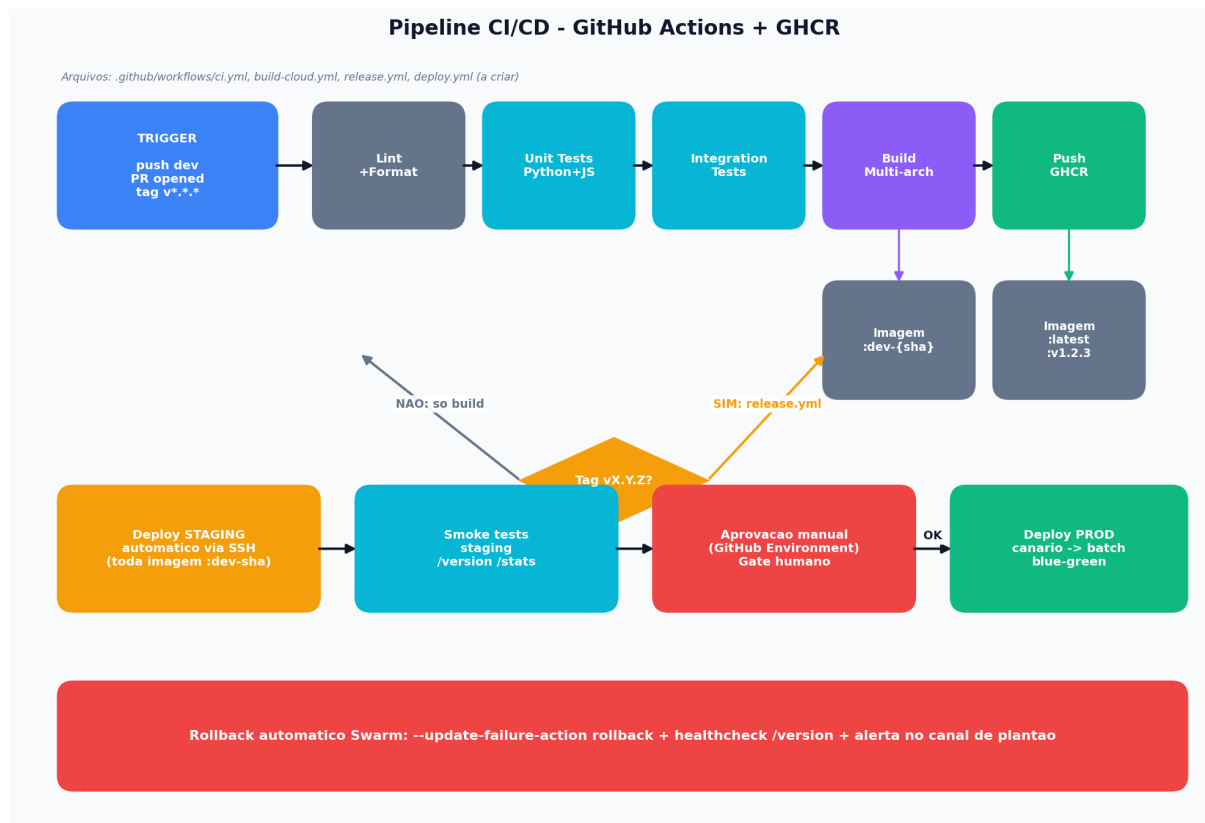


Figura 3 - Pipeline completo: trigger -> testes -> build -> deploy

5.1 Workflows existentes

Arquivo	Trigger	Acao
.github/workflows/ci.yml	PR e push em qualquer branch	Lint + unit tests + integration tests
.github/workflows/build-cloud.yml	push em dev, tag v*	Build multi-arch (AMD64, ARM64, RPi, Jetson) + push GHCR
.github/workflows/release.yml	tag v*	Re-tag para versao stable/beta + GitHub Release

5.2 Workflow de deploy a criar (deploy.yml)

Hoje o deploy em STAGING e PROD e manual via SSH. Para reduzir erro humano, criar .github/workflows/deploy.yml com dois jobs:

Job 1: deploy-staging - automatico em push em dev

- Conecta via SSH no host de staging
- Atualiza tag da imagem para :dev-{sha}
- Executa `docker stack deploy`
- Aguarda healthcheck (`curl /version`)

- Roda smoke tests basicos (3-5 endpoints)
- Notifica canal Slack/Discord

Job 2: deploy-prod - manual com aprovacao

- Trigger: criacao de tag v*.*.* em main
- Usa GitHub Environment production com required reviewers
- Aguarda aprovacao manual de **2 mantenedores**
- Executa deploy canario -> batch -> 100% (ver Cap. 7)
- Em caso de falha de healthcheck, faz rollback automatico
- Notifica canal e atualiza status page

Por que GitHub Environment com required reviewers? Isso força um humano a clicar 'Approve' antes do deploy em prod, mesmo com tudo automatizado. É o 'gate humano' que diferencia automacao de irresponsabilidade.

5.3 Secrets necessarios

Secret	Onde	Uso
GHCR_TOKEN	Repo	Push de imagem no GHCR (ja existe)
STAGING_SSH_KEY	Repo	SSH para o VPS de staging
STAGING_HOST	Repo	Hostname/IP do VPS staging
PROD_SSH_KEY	Environment 'production'	SSH para o VPS prod
PROD_HOST	Environment 'production'	Hostname/IP do VPS prod
SLACK_WEBHOOK	Repo	Notificacoes de deploy
SENTRY_AUTH_TOKEN	Repo	Upload de sourcemaps + release tracking

6. Estrategia de Testes

Cinco niveis de teste, do mais barato/rapido ao mais caro/lento. Todos rodam no CI antes do merge em dev.

Nivel	Onde	Tempo	Cobertura alvo	Quando roda
1. Lint + Format	ruff, prettier	< 30 s	100% dos arquivos	Em todo PR
2. Unit (Python)	pytest	2-5 min	> 70% codigo	Em todo PR
3. Unit (JS)	vitest	1-2 min	> 60% codigo	Em todo PR
4. Integration	pytest + fixtures	5-10 min	Fluxos criticos	Em todo PR
5. Smoke (E2E)	playwright + curl	3-5 min	5-10 fluxos chave	Apos deploy staging
6. Carga (opcional)	k6 / locust	30 min	API + WebSocket	Antes de release maior

6.1 Testes minimos por dominio

Camera / Stream

Testes que cobrem: reconexao apos perda de stream, frame nulo, decoder com codec invalido, camera offline. *Onde quebrou antes, sempre tem teste.*

Detection / IA

Testes que cobrem: zonas configuradas corretamente, label-map valido, confidence threshold, modelo carregado. Mock do detector para nao depender de hardware.

Recording / S3 sync

Testes que cobrem: gravacao iniciada, segment closing, upload S3 retry, falha de credencial, espaco em disco. Mock do boto3.

Auth / Licensing

Testes que cobrem: token expirado, license invalida, heartbeat falha, fallback offline. Critico para nao bloquear o cliente.

Migrations

Para cada migration, teste de up + down + idempotencia. Ver capitulo 10.

6.2 Smoke tests obrigatorios em staging

Estes 8 testes **sempre** rodam apos deploy em staging. Falha em qualquer um aborta a promocao para producao.

#	Teste	Endpoint / acao	Criterio
1	Versao	GET /version	Retorna a tag esperada
2	Healthcheck	GET /api/version	200 OK em < 2s
3	Login	POST /api/login	200 + cookie de sessao

4	Lista cameras	GET /api/config	JSON valido + cameras
5	Snapshot	GET /api/{cam}/snapshot.jpg	JPEG > 0 bytes
6	Eventos	GET /api/events?limit=5	Array (pode vazio)
7	Stats	GET /api/stats	JSON com cpu/mem
8	WS Live	WS /ws	Handshake completa

7. Procedimento de Deploy

Deploy em producao usa estrategia **canario + rolling update blue-green**. Quatro fases sequenciais com gates humanos entre elas.

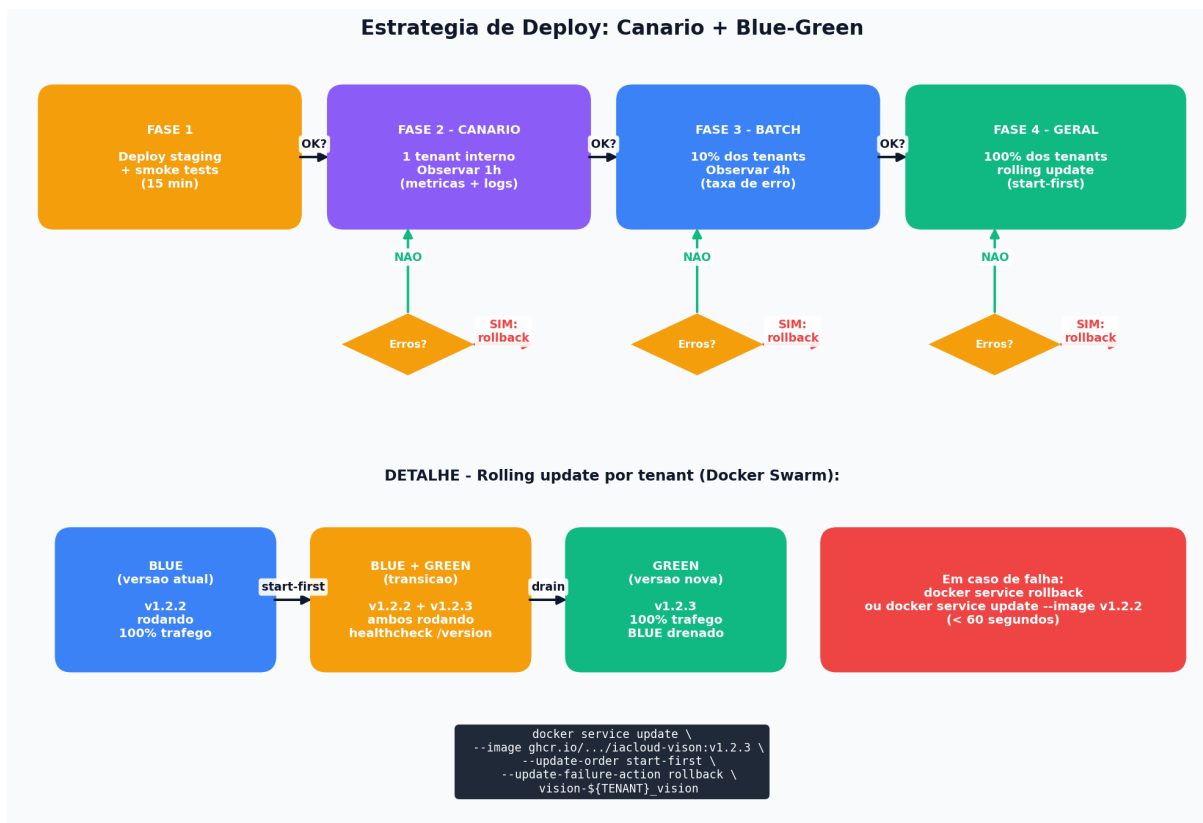


Figura 4 - Fases do deploy: staging -> canario -> batch -> 100%

7.1 Resumo das 4 fases

Fase	Escopo	Duracao	Gate
1 - Staging	1 ambiente isolado	15 min observacao	Smoke tests passam
2 - Canario	1 tenant interno	1 h observacao	Zero erro 5xx
3 - Batch	10% dos tenants	4 h observacao	Erro < 0.1%
4 - Geral	100% restante	Rolling update	Healthcheck OK

7.2 Comando-chave do Docker Swarm

Toda atualizacao de tenant em producao usa este comando. Os flags estao explicados abaixo:

```
docker service update \
  --image ghcr.io/tarcisioazevedo/iacloud-vision:vX.Y.Z \
  --update-order start-first \
  --update-failure-action rollback \
  --update-monitor 60s \
  --update-parallelism 1 \
```

```
vision-${TENANT}_vision
```

Por que cada flag:

- **--image vX.Y.Z**: Tag versionada (nao :latest) - garante rastreabilidade.
- **--update-order start-first**: Sobe o container novo ANTES de matar o velho - evita downtime.
- **--update-failure-action rollback**: Se healthcheck falhar, Swarm reverte sozinho - rede de seguranca.
- **--update-monitor 60s**: Aguarda 60s observando saude apos cada container atualizado.
- **--update-parallelism 1**: Atualiza um por vez (mesmo dentro de um stack com multiplas replicas).

7.3 Janela de manutencao

Mesmo com downtime de 10-30 segundos, comunicar tenants ajuda:

- Aviso 7 dias antes para releases maiores
- Aviso 24h antes para releases menores
- Sem aviso para hotfixes (urgencia justifica)
- Janela preferencial: **terca a quinta, 14h-16h BRT** (maior disponibilidade da equipe)
- ~~Nunca: sexta a noite, finais de semana, vesperas de feriado~~

Regra de Murphy: a 1 hora antes da virada do ano, alguem vai querer empurrar um deploy 'rapidinho'.
NAO. A janela existe pra isso - respeite.

8. Procedimento de Rollback

Rollback e o ultimo recurso, mas tem que estar pronto sempre. Dois caminhos: simples (sem mexer no DB) e com restore.

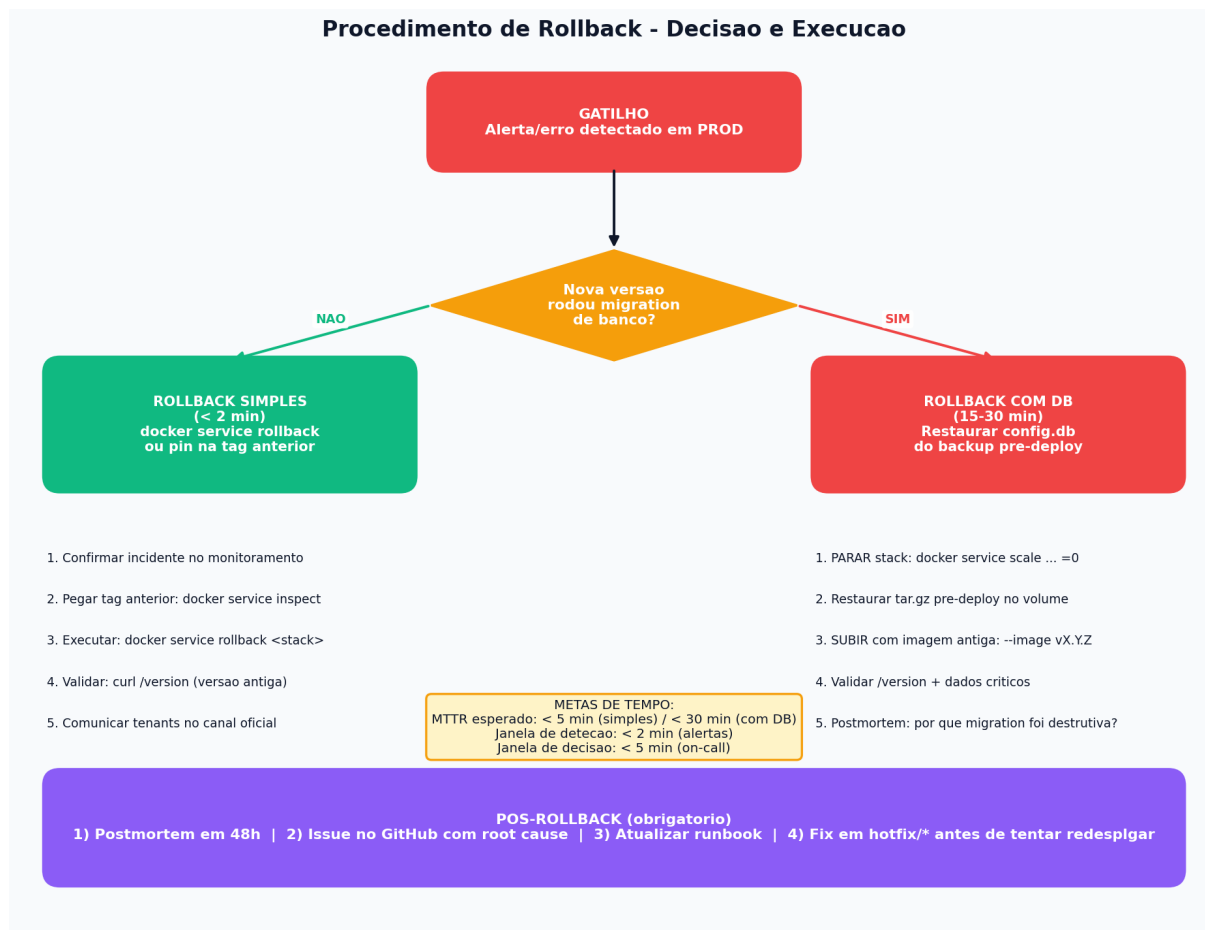


Figura 5 - Fluxograma de decisao do rollback

8.1 Quando fazer rollback

Disparar rollback se **qualquer um** destes critrios for atingido apos o deploy:

- Taxa de erro 5xx > 1% por mais de 2 minutos
- Latencia p95 > 2x baseline por mais de 5 minutos
- Endpoint /version retornando 5xx
- Mais de 1 reclamacao de tenant por canal oficial em 30 min
- Volume de disco crescendo > 10x baseline (vazamento)
- Container restartando (CrashLoop)

8.2 Caminho A - Rollback simples (< 2 min)

Quando a versao nova **nao** rodou migration de banco. O Swarm guarda a tag anterior automaticamente.

```
# Volta para a versao anterior do mesmo servico
docker service rollback vision-${TENANT}_vision

# Validar
```

```
curl -s https://vision.${TENANT}.iacv.com/version
docker service ps vision-${TENANT}_vision
```

8.3 Caminho B - Rollback com restore (15-30 min)

Quando a versao nova rodou migration. Requer parar o servico, restaurar o backup pre-deploy, e subir versao antiga.

Preparacao obrigatoria: backup pre-deploy precisa existir (Fase 0 do Runbook 01).

```
# 1. Parar
docker service scale vision-${TENANT}_vision=0

# 2. Restaurar volume
docker run --rm -v vision-${TENANT}_config:/data \
-v /backups/predeploy:/backup:ro alpine \
sh -c "rm -rf /data/* && \
tar xzf /backup/${TENANT}-predeploy.tar.gz -C /"

# 3. Subir versao antiga
docker service update \
--image ghcr.io/.../iaccloud-vision:vX.Y.Z-1 \
vision-${TENANT}_vision
docker service scale vision-${TENANT}_vision=1
```

8.4 Detalhes completos

O procedimento passo-a-passo (incluindo postmortem obrigatorio) esta no **Runbook 02** ao final deste documento.

9. Procedimento de Hotfix

Hotfix e para quando rollback nao resolve e nao da pra esperar o proximo release. Processo otimizado para velocidade **sem perder rigor**.

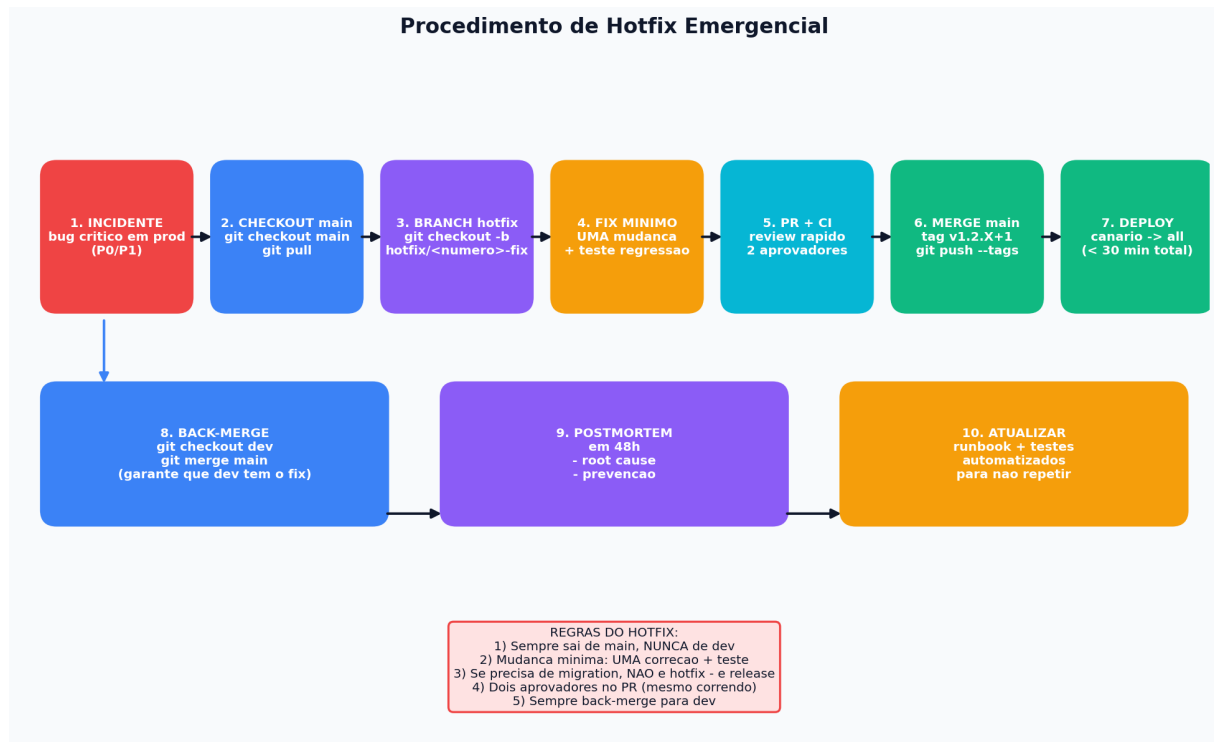


Figura 6 - Fluxo de hotfix: 10 passos obrigatorios

9.1 Criterios para hotfix (todos obrigatorios)

- Severidade P0 ou P1 (impacto significativo aos tenants)
- Rollback nao resolve (versao antiga tem o mesmo bug)
- Fix cabe em <= 50 linhas de codigo
- **Nao** requer alteracao de schema do banco
- Tem (ou consegue criar em < 10 min) teste de regressao

Se algum criterio falhar, NAO use hotfix. Faca rollback (Cap. 8) e planeje fix no proximo release. Hotfix com escopo errado e a maior fonte de incidentes em sequencia.

9.2 Passos resumidos

1. **Setup:** `git checkout main && git pull && git checkout -b hotfix/v1.2.4-fix-X`
2. **Fix minimo:** Edita arquivo + adiciona teste regression. UMA mudanca.
3. **PR:** `gh pr create --base main --label hotfix`
4. **Review:** **2 reviewers** (curto mas obrigatorio)
5. **Tag:** `git tag v1.2.4 && git push --tags` (dispara build automatico)
6. **Backup:** Backup rapido de config.db de todos tenants
7. **Canario:** Deploy em 1 tenant interno, observar 15 min
8. **Geral:** Deploy em todos os tenants (rolling)

9. Back-merge: `git checkout dev && git merge main && git push`

10. Postmortem: Em 48h: timeline + root cause + prevencao

9.3 Tempo total esperado

Da deteccao do bug ate 100% dos tenants atualizados: **30-60 minutos**. Se passar de 90 minutos, considere rollback enquanto o fix amadurece.

10. Migrations de Banco

O IACV roda **35+ migrations** Peewee no startup do container. Esta e a area de maior risco em deploys - merece tratamento especial.

10.1 Padrao expand-and-contract

Toda mudanca de schema potencialmente destrutiva (rename de coluna, drop de tabela, mudanca de tipo) e dividida em **tres releases**:

Release	Migration	Codigo da app	Estado do banco
N (expand)	ADD coluna nova (sem dropar a antiga)	Le da nova OU antiga. Escreve em ambas.	Coluna velha + nova (redundante)
N+1 (migrate)	Backfill (copia dados velhos -> novos)	Le da nova. Escreve em ambas.	Ambas com mesmos dados
N+2 (contract)	DROP coluna velha	Le e escreve so na nova.	So coluna nova

10.2 Por que esse trabalho extra?

- **Rollback fica seguro:** a release N pode voltar em qualquer momento - o banco ainda tem a coluna antiga.
- **Zero downtime:** nunca ha um momento em que codigo e banco estao incompativeis.
- **Observabilidade:** entre N e N+2 voce mede: quantas leituras ainda usam a coluna antiga? Se houver, tem bug ainda nao corrigido.

10.3 Checklist antes de qualquer migration

- [] Migration tem teste up + down + idempotencia (rodar 2x sem erro)
- [] Backup pre-deploy validado (descompacta sem erro)
- [] Migration foi rodada em staging com volume de dados similar
- [] Tempo de execucao medido em staging (deve ser < 30s)
- [] Plano de rollback documentado (Caminho A ou B)
- [] Se DROP/RENAME: split em 3 releases (expand-and-contract)

Regra de bolso: migration que demora > 1 minuto em producao e candidata a problema. Se a tabela tem milhoes de linhas, faca o backfill em batches via job assincrono - nao na migration.

11. Monitoramento e Alertas

Sem observabilidade, deploy e roleta russa. Stack minimo recomendado:

Camada	Ferramenta	O que monitora
Metricas	Prometheus + Grafana	CPU, memoria, taxa de erro, latencia, queue size, FPS por camera
Logs	Loki + Grafana	Logs de containers (centralizados) busca por tenant/severity
Erros app	Sentry	Stack traces Python e React, release tracking
Uptime	Uptime Kuma ou BetterStack	Pings externos /version a cada 30s por tenant
Alertas	Alertmanager + Telegram/WhatsApp	Disparos para on-call com escalation

11.1 Alertas obrigatorios (P0/P1)

Alerta	Threshold	Severidade	Acao
Tenant down	/version 5xx > 1 min	P0	Acionar plantao
Erro alto	5xx > 1% por 5 min	P0	Acionar plantao
Latencia alta	p95 > 5s por 10 min	P1	Investigar
Disco cheio	> 90%	P1	Limpar / aumentar
Camera offline	> 2 min sem frame	P2	Notificar tenant
License expira	< 7 dias	P2	Renovar

11.2 Resposta a incidente

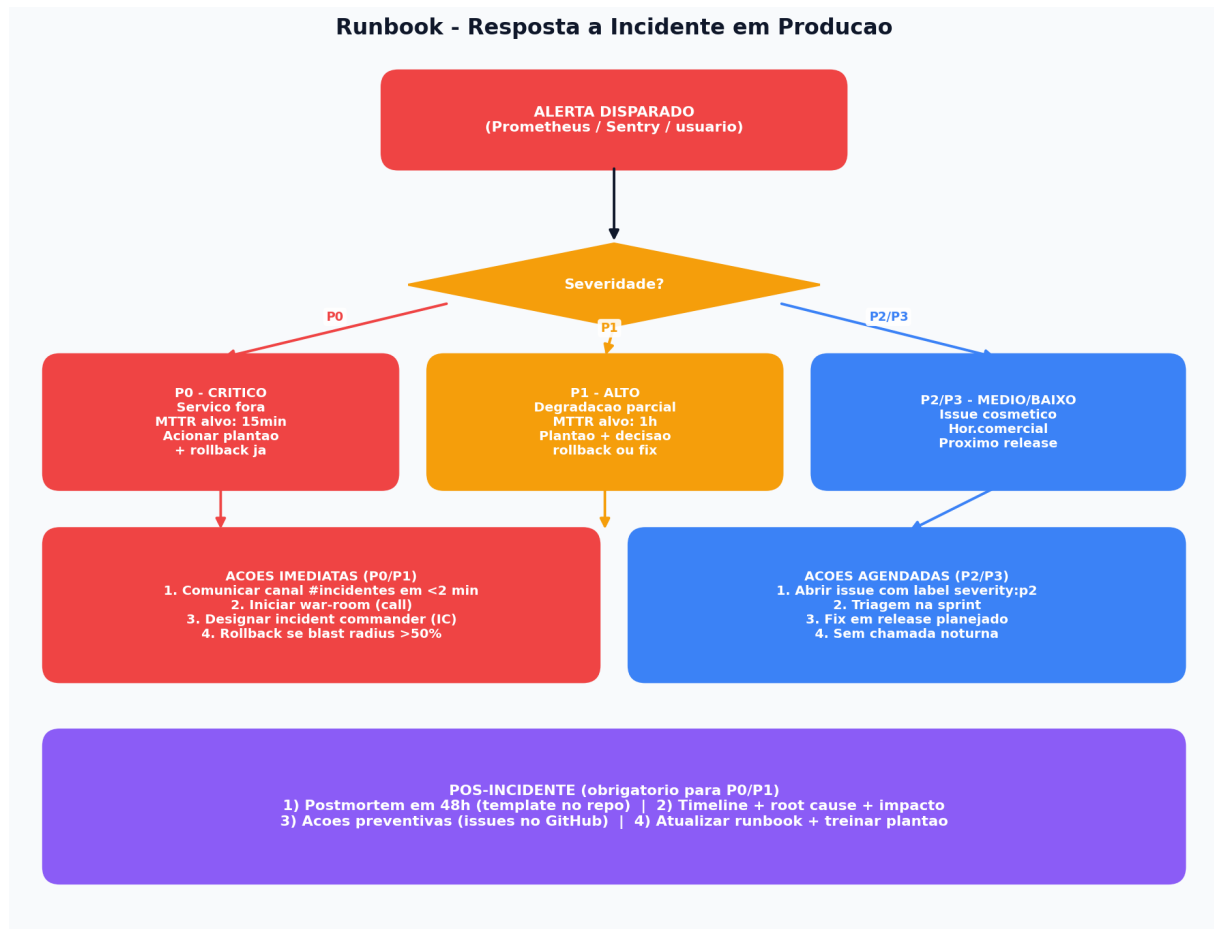


Figura 7 - Fluxo de resposta a incidente em producao

12. Backup e Disaster Recovery

Backup que nunca foi testado nao e backup - e esperanca. Politica resumida abaixo, detalhes no Runbook 04.

Dado	Frequencia	Onde	Retencao	Restore tested
config.db	1 hora + pre-deploy	Local + S3	30 dias rolling 90 dias predeploy	Mensal
Recordings	Continuo (S3 sync)	S3 (Backblaze/AWS)	90 dias	Trimestral
Snapshots	Continuo	S3	30 dias	Trimestral
Volumes Docker	Snapshot semanal (Btrfs / ZFS)	Disco secundario	4 semanas	Trimestral
Configs Traefik + Letsencrypt	Pre-deploy	S3	Indefinido	Em DR test

12.1 Metas RPO/RTO

- **RPO (perda maxima aceitavel):** 1 hora (config.db) / 5 minutos (recordings em S3 sync)
- **RTO (tempo de restauracao):** 30 minutos (1 tenant) / 4 horas (DR completo de host)

12.2 Teste de DR (trimestral - obrigatorio)

A cada 3 meses, fazer um teste real de Disaster Recovery: provisionar VPS novo, restaurar do S3, validar que tudo funciona. Documentar tempo total. Se passar do RTO, e bug - tem que melhorar o processo.

13. Plano de Implementacao (90 dias)

Nem tudo precisa ser feito de uma vez. Ordem sugerida para sair do estado atual e chegar em missao critica madura:

Sprint	Foco	Entregaveis
S1-2 (sem 1-2)	Branch protection + versionamento	Branch rules main e dev, CONTRIBUTING.md, CHANGELOG, primeira tag v1.0.0
S3-4	Backup automatico	Cron de backup horario, upload S3, teste de restore
S5-6	Staging real	VPS staging provisionado, deploy.yml automatizado, smoke tests de 8 endpoints
S7-8	Observabilidade	Prometheus + Grafana, Loki para logs, Sentry no Python e React
S9-10	Alertas P0/P1	Alertmanager configurado, on-call definido, runbooks impressos
S11-12	Deploy canario	Workflow de canario, gate humano em GH Environment, primeiro deploy controlado
S13	Primeiro DR test	Restaurar host inteiro do S3, medir RTO real, ajustar runbook

Atalho: se tiver pressa, faca pelo menos S1-2 + S3-4 no primeiro mes. Branch protection + backup ja tira 80% do risco de pe. O resto pode ir maturando.

14. Runbooks Detalhados

Os runbooks abaixo sao documentos vivos. Toda vez que houver incidente, atualize-os. Imprima e cole na parede da sala da operacao.

Runbook 01 - Deploy em Producao

Quando usar: liberar uma nova versão (vX.Y.Z) para os tenants em produção.

MTTR alvo: 30–60 min (com canário). **Downtime por tenant:** 10–30 s.

Pré-requisitos

- [] PR mergeado em dev e release/vX.Y.Z criado
- [] CI verde em todos os jobs (lint, unit, integration, build multi-arch)
- [] Imagem ghcr.io/tarcisioazevedo/iacloud-vision:vX.Y.Z publicada no GHCR
- [] CHANGELOG.md atualizado com a versão
- [] Janela de deploy combinada com tenants críticos (se aplicável)
- [] Acesso SSH ao host de produção
- [] Backups recentes de todos os tenants (< 24 h)

Fase 0 — Backup preventivo (5 min)

Em produção, **antes de qualquer coisa**, backup de todos os volumes:

```
# Para cada tenant
for TENANT in $(docker stack ls --format '{{.Name}}' | grep '^vision-'); do
  docker run --rm \
  -v ${TENANT}_config:/data:ro \
  -v /backups/predeploy:/backup \
  alpine tar czf /backup/${TENANT}-(date +%Y%m%d-%H%M).tar.gz /data
done

# Validar
ls -lh /backups/predeploy/ | tail -20
```

Checkpoint: todos os tenants têm um .tar.gz recente. Se algum falhar, **PARE** o deploy.

Fase 1 — Deploy em STAGING (15 min)

```
# Atualizar tag no compose de staging
ssh staging-host
cd /opt/iacv-staging
sed -i "s|iacloud-vision:.*/iacloud-vision:vX.Y.Z|" docker-compose.staging.yml

# Aplicar
docker stack deploy -c docker-compose.staging.yml iacv-staging

# Aguardar healthcheck
watch -n 5 'docker service ls | grep iacv-staging'
```

Smoke tests:

```
# 1. Versão correta
curl -s https://staging.iacv.example.com/version
# esperado: vX.Y.Z

# 2. Stats acessível (autenticado)
```

```
curl -su admin:senha https://staging.iacv.example.com/stats | jq .

# 3. Login funciona
curl -X POST https://staging.iacv.example.com/api/login \
-d '{"user":"admin","password":"..."}' \
-H 'Content-Type: application/json'

# 4. Câmera de teste retornando snapshot
curl -so /tmp/snap.jpg https://staging.iacv.example.com/api/camera_test/snapshot.jpg
file /tmp/snap.jpg # deve ser JPEG válido
```

Checkpoint: todos os 4 testes OK. Se algum falhar, **PARE** e investigue.

Manter staging rodando por **mínimo 1 h** observando:

- `docker service logs -f iacv-staging_vision`
- Sentry / Grafana — taxa de erro deve ser zero

Fase 2 — Deploy CANÁRIO (1 tenant interno, 1h)

Escolher **1 tenant interno** (de testes/demo) — nunca cliente real no canário.

```
ssh prod-host
TENANT=demo # tenant interno

docker service update \
--image ghcr.io/tarcisioazevedo/iacloud-vision:vX.Y.Z \
--update-order start-first \
--update-failure-action rollback \
--update-monitor 60s \
vision-${TENANT}_vision
```

Validações imediatas:

```
# Versão
curl -s https://vision.${TENANT}.iacv.example.com/version

# Logs por 10 min
docker service logs -f --since 10m vision-${TENANT}_vision

# Métricas
# Acessar Grafana: https://grafana.iacv.example.com/d/iacv
# Verificar painéis: error rate, latência p95, CPU, RAM
```

Manter rodando por 1 h. Critérios de sucesso:

- Erro 5xx por minuto: 0
- Latência p95: dentro do baseline (verificar Grafana)
- Logs: sem `ERROR` ou `CRITICAL` recorrentes
- Câmeras: detectando objetos normalmente
- Recordings: gravando

Se algum critério falhar → **rollback** (Runbook 02).

Fase 3 — Deploy BATCH (10% dos tenants, 4h)

Selecionar 10% dos tenants (preferir os menos críticos):

```
TENANTS_BATCH="tenant_a tenant_b tenant_c" # 10% da lista total

for TENANT in $TENANTS_BATCH; do
echo "Atualizando ${TENANT}..."
docker service update \
--image ghcr.io/tarcisioazevedo/iacloud-vision:vx.Y.Z \
--update-order start-first \
--update-failure-action rollback \
--update-monitor 60s \
vision-${TENANT}_vision

# Aguardar healthcheck antes do próximo
sleep 30
curl -fs https://vision.${TENANT}.iacv.example.com/version || {
echo "FALHA em ${TENANT} – abortando"
exit 1
}
done
```

Manter rodando por 4 h. Mesmos critérios da Fase 2.

Fase 4 — Deploy GERAL (100% dos tenants)

Após 4 h de batch sem incidentes:

```
ALL_TENANTS=$(docker stack ls --format '{{.Name}}' | grep '^vision-' | sed 's/^vision-//; s/_vision$//')

for TENANT in $ALL_TENANTS; do
# Pular os já atualizados
CURRENT=$(curl -fs https://vision.${TENANT}.iacv.example.com/version)
[ "$CURRENT" = "vx.Y.Z" ] && continue

echo "Atualizando ${TENANT}..."
docker service update \
--image ghcr.io/tarcisioazevedo/iacloud-vision:vx.Y.Z \
--update-order start-first \
--update-failure-action rollback \
--update-monitor 60s \
vision-${TENANT}_vision

sleep 30
done
```

Fase 5 — Pós-deploy

- [] Tag vx.Y.Z marcada como latest no GHCR (release.yml já faz)
- [] CHANGELOG.md publicado em /CHANGELOG.md
- [] Comunicação aos tenants no canal oficial
- [] Postmortem se houve algum hiccup
- [] Backups da Fase 0 mantidos por **mínimo 7 dias**
- [] Imagem vx.Y.Z-1 (anterior) mantida acessível

Critérios de aborto

ABORTAR e fazer rollback se:

- Taxa de erro > 1% em qualquer tenant atualizado
- Latência p95 > 2x baseline
- Healthcheck `/version` retornando 5xx
- Volume de disco crescendo anormalmente (vazamento)
- Mais de 1 reclamação de cliente em 30 min

Ver **Runbook 02 — Rollback**.

Runbook 02 - Rollback de Producao

Quando usar: quando uma versão recém-deployada apresenta problema em produção.

MTTR alvo: < 5 min (rollback simples) / < 30 min (com restore de DB).

Decisão: qual tipo de rollback?

A nova versão rodou MIGRATION de banco?

■

■■■ NÃO → ROLLBACK SIMPLES (docker service rollback)

■

■■■ SIM → ROLLBACK COM RESTORE DE BANCO

Como saber se houve migration?

```
# Compare o número de migrations entre as imagens
docker run --rm ghcr.io/tarcisioazevedo/iacloud-vision:vX.Y.Z-1 \
ls /opt/frigate/migrations | wc -l

docker run --rm ghcr.io/tarcisioazevedo/iacloud-vision:vX.Y.Z \
ls /opt/frigate/migrations | wc -l
```

Se os números forem diferentes → **rollback com DB**.

Caminho A — Rollback simples (< 2 min)

```
ssh prod-host
TENANT=tenant_afetado

# Opção 1: rollback automático do Swarm (volta pra imagem anterior)
docker service rollback vision-${TENANT}_vision

# Opção 2: forçar uma tag específica
docker service update \
--image ghcr.io/tarcisioazevedo/iacloud-vision:vX.Y.Z-1 \
--update-order start-first \
vision-${TENANT}_vision
```

Validar:

```
# Versão voltou
curl -s https://vision.${TENANT}.iacv.example.com/version

# Healthcheck OK
docker service ps vision-${TENANT}_vision | head -5

# Logs limpos
docker service logs --tail 50 vision-${TENANT}_vision
```

Se múltiplos tenants foram afetados, rodar o loop:

```
TENANTS_AFETADOS="tenant_a tenant_b tenant_c"
for TENANT in $TENANTS_AFETADOS; do
docker service rollback vision-${TENANT}_vision
done
```

Caminho B — Rollback com restore de banco (15–30 min)

ATENÇÃO: este procedimento causa downtime de ~5 min por tenant.

B.1 — Preparar

```
ssh prod-host
TENANT=tenant_afetado

# Localizar o backup pré-deploy (Fase 0 do Runbook 01)
ls -lh /backups/predeploy/vision-${TENANT}/*.tar.gz | tail -3

# Confirmar ID do backup correto
BACKUP=/backups/predeploy/vision-${TENANT}-20260427-1430.tar.gz
```

B.2 — Parar o serviço

```
docker service scale vision-${TENANT}_vision=0
sleep 10
docker service ps vision-${TENANT}_vision # confirmar que parou
```

B.3 — Restaurar o volume

```
# Backup do estado corrompido (caso precise para postmortem)
docker run --rm \
-v vision-${TENANT}_config:/data:ro \
-v /backups/corrupted:/backup \
alpine tar czf /backup/vision-${TENANT}-corrupted-$(date +%Y%m%d-%H%M).tar.gz /data

# Limpar volume
docker run --rm \
-v vision-${TENANT}_config:/data \
alpine sh -c 'rm -rf /data/*'

# Restaurar
docker run --rm \
-v vision-${TENANT}_config:/data \
-v /backups/predeploy:/backup:ro \
alpine tar xzf /backup/${basename $BACKUP} -C /
```

B.4 — Subir com versão antiga

```
docker service update \
--image ghcr.io/tarcisioazevedo/iacloud-vision:vX.Y.Z-1 \
vision-${TENANT}_vision

docker service scale vision-${TENANT}_vision=1
```

B.5 — Validar

```
# Versão
curl -s https://vision.${TENANT}.iacv.example.com/version
# Esperado: vX.Y.Z-1

# Dados críticos: cameras retornando snapshot
curl -fso /tmp/snap.jpg
```

```
https://vision.${TENANT}.iacv.example.com/api/<camera>/snapshot.jpg
file /tmp/snap.jpg

# Recordings recentes
curl -fs https://vision.${TENANT}.iacv.example.com/api/events?limit=5 | jq .
```

Pós-rollback (obrigatório)

Imediato (primeiros 30 min)

- [] Comunicar tenants no canal oficial
- [] Atualizar status page com incidente
- [] Snapshot de logs e métricas do momento da falha
- [] Issue no GitHub com label `incident:p0` ou `p1`

Em 48 h

- [] **Postmortem completo** seguindo template em `/docs/templates/postmortem.md`:
- Timeline detalhada (UTC)
- Root cause (5 whys)
- Impacto (tenants afetados, duração, SLA)
- O que funcionou bem na resposta
- O que falhou
- Ações preventivas (cada uma vira issue no GitHub)
- [] **Não tentar redeploy da versão problemática** até:
 - Root cause identificado
 - Fix em hotfix branch + testes adicionados
 - Smoke tests cobrindo o cenário do incidente
- [] **Atualizar este runbook** se descobriu cenário não coberto

Comandos de emergência (cola rápida)

```
# Rollback de um tenant
docker service rollback vision-<TENANT>_vision

# Rollback de TODOS os tenants (situação crítica)
for s in $(docker service ls --format '{{.Name}}' | grep '^vision-'); do
  docker service rollback "$s"
done

# Listar versões em execução
for s in $(docker service ls --format '{{.Name}}' | grep '^vision-'); do
  echo -n "$s: "
  docker service inspect "$s" --format '{{.Spec.TaskTemplate.ContainerSpec.Image}}'
done

# Forçar uma tag específica
docker service update --image ghcr.io/.../iacloud-vision:vX.Y.Z vision-<TENANT>_vision
```

Runbook 03 - Hotfix Emergencial

Quando usar: bug crítico em produção que **NÃO** pode esperar pelo próximo release.

MTTR alvo: < 30 min do diagnóstico ao deploy.

***NÃO use hotfix se:** o problema requer migration de banco, refactor maior, ou se um rollback resolve. Hotfix é para fix mínimo e cirúrgico.*

Critérios para hotfix

Use o caminho de hotfix **somente se TODOS** estes pontos forem verdadeiros:

- [] Severidade P0 ou P1 (impacto significativo aos tenants)
- [] Rollback **não** é opção (versão antiga também tem o bug, ou seria perda de feature crítica)
- [] Fix cabe em **≤ 50 linhas de código**
- [] Não requer alteração de schema do banco
- [] Existe (ou consegue criar em < 10 min) um teste de regressão

Se algum item for falso → use **Runbook 02 — Rollback** + planejar fix no próximo release.

Fase 1 — Setup (3 min)

```
# Estar em main atualizado (NUNCA partir de dev em hotfix)
git checkout main
git pull --ff-only origin main

# Confirmar tag atual em produção
git describe --tags
# exemplo: v1.2.3

# Criar branch hotfix
git checkout -b hotfix/v1.2.4-camera-stream-crash
```

Fase 2 — Fix mínimo (15 min)

Regra: uma mudança, um arquivo idealmente, com teste regression.

```
# Editar APENAS o necessário
$EDITOR frigate/camera_stream.py

# Adicionar teste que falha sem o fix
$EDITOR tests/test_camera_stream.py

# Rodar testes localmente
make test
# ou
pytest tests/test_camera_stream.py -v
```

Commit:

```
git add frigate/camera_stream.py tests/test_camera_stream.py
git commit -m "fix(camera): handle null frame in stream restart"
```


Quando a câmera reinicia o stream sem frame inicial, evita NPE no decoder. Adiciona teste regression.

Refs: incident #42
Severity: P0"

Fase 3 — PR + CI (5–10 min)

```
git push -u origin hotfix/v1.2.4-camera-stream-crash

# Criar PR (gh CLI)
gh pr create \
  --base main \
  --title "[HOTFIX] fix(camera): handle null frame in stream restart" \
  --body "$(cat <<'EOF'
## Severidade
P0 – câmeras travando quando há perda de stream

## Causa
Decoder recebe `None` no frame após restart, causando crash no thread de processamento.

## Fix
Validar `frame is not None` antes de decode.

## Teste
- [x] Unit test adicionado em `tests/test_camera_stream.py`
- [x] Reproduzido localmente com mock
- [x] CI verde

## Plano de deploy
Deploy direto em produção via canário (1 tenant interno → batch → 100%).
Tempo estimado: 25 min.
EOF
)" \
  --label "hotfix" --label "severity:p0"
```

Aprovação: 2 reviewers obrigatórios mesmo em emergência (mas review rápido — < 5 min).

Fase 4 — Tag + Release (2 min)

Após PR mergeado em `main`:

```
git checkout main
git pull --ff-only origin main

# Tag patch (incremento do PATCH)
git tag -a v1.2.4 -m "Hotfix v1.2.4 – camera stream null frame"
git push origin v1.2.4
```

O `release.yml` builda automaticamente a imagem `ghcr.io/.../iaccloud-vision:v1.2.4` (5–8 min).

Acompanhe:

```
gh run list --workflow=release.yml --limit 5
gh run watch
```

Fase 5 — Deploy emergencial (10 min)

Modo emergencial: canário reduzido (15 min) ao invés de 1 h.

```
ssh prod-host

# 1. Backup rápido (5 min)
for TENANT in $(docker stack ls --format '{{.Name}}' | grep '^vision-'); do
  docker run --rm \
  -v ${TENANT}_config:/data:ro \
  -v /backups/hotfix:/backup \
  alpine tar czf /backup/${TENANT}-${date +%Y%m%d-%H%M}.tar.gz /data &
done
wait

# 2. Canário em 1 tenant interno (15 min observação)
docker service update \
--image ghcr.io/tarcisioazevedo/iacloud-vision:v1.2.4 \
--update-order start-first \
--update-failure-action rollback \
vision-demo_vision

# Aguardar e verificar
sleep 60
curl -s https://vision.demo.iacv.example.com/version # v1.2.4
docker service logs --tail 50 vision-demo_vision

# 3. Se OK em 15 min → deploy em todos
sleep 900 # 15 min de observação

ALL_TENANTS=$(docker stack ls --format '{{.Name}}' | grep '^vision-' | sed 's/^vision-//;
s/_vision$//')
for TENANT in $ALL_TENANTS; do
  docker service update \
  --image ghcr.io/tarcisioazevedo/iacloud-vision:v1.2.4 \
  --update-order start-first \
  --update-failure-action rollback \
  vision-${TENANT}_vision
  sleep 20
done
```

Fase 6 — Back-merge (3 min)

OBRIGATÓRIO: garantir que dev também tem o fix.

```
git checkout dev
git pull --ff-only origin dev
git merge main --no-ff -m "chore: back-merge hotfix v1.2.4 into dev"
git push origin dev
```

Sem isso, o próximo release **regride o fix** (catástrofe).

Fase 7 — Postmortem (em 48 h)

- [] Postmortem escrito (template `/docs/templates/postmortem.md`)
- [] Issue de prevenção criada com label `prevention`
- [] Teste adicional cobrindo cenário (se faltava)

- [] Atualizar este runbook se descobriu lacuna no processo

Anti-patterns (NÃO faça)

Anti-pattern	Por quê	O que fazer
Hotfix com 200+ linhas	Aumenta risco de novo bug	Rollback + release planejado
Hotfix incluindo nova feature	"Já que estamos mexendo..."	Hotfix é só o fix
Pular review por urgência	2 olhos pegam bobagem	2 reviewers em < 5 min
Deploy direto sem canário	Risco de derrubar todos	Canário de 15 min mínimo
Esquecer back-merge para dev	Regressão no próximo release	Sempre back-merge
Hotfix com migration de DB	Não é hotfix, é release	Restore de backup + release planejado

Comandos rápidos (cola)

```
# Hotfix em 1 comando (situação real)
git checkout main && git pull && \
git checkout -b hotfix/$(date +%Y%m%d)-fix-XYZ && \
$EDITOR frigate/file.py && \
make test && \
git add -A && git commit -m "fix: ..." && \
git push -u origin HEAD && \
gh pr create --base main --label hotfix
```

Runbook 04 - Backup e Restore

Quando usar: rotina diária de backup, antes de cada deploy, e em caso de DR (Disaster Recovery).

RPO alvo: 1 hora. **RTO alvo:** 30 minutos.

O que é guardado em backup

Por tenant, três classes de dados:

Classe	Volume	Frequência	Retenção	Onde
config.db (SQLite)	`vision-\${TENANT}_config`	A cada 1h	30 dias	S3 + local
Recordings (video)	`vision-\${TENANT}_media`	Contínuo (S3 sync)	90 dias	S3
Snapshots (jpg)	`vision-\${TENANT}_media`	Contínuo (S3 sync)	30 dias	S3

A app **já tem** sync para S3 (`frigate/s3_sync.py`). O que falta é o backup periódico do `config.db`.

Backup horário do config.db (cron)

Criar o script `/opt/iacv/scripts/backup-config.sh`:

```
#!/usr/bin/env bash
set -euo pipefail

BACKUP_DIR=/backups/hourly
S3_BUCKET=s3://iacv-backups/config-db
RETENTION_DAYS=30

mkdir -p "$BACKUP_DIR"
TS=$(date +%Y%m%d-%H%M%S)

for TENANT in $(docker stack ls --format '{{.Name}}' | grep '^vision-'); do
    OUT="$BACKUP_DIR/${TENANT}-${TS}.tar.gz"

    docker run --rm \
        -v ${TENANT}_config:/data:ro \
        -v "$BACKUP_DIR":/backup \
        alpine tar czf "/backup/${basename ${OUT}}/" /data 2>/dev/null

    # Upload S3
    aws s3 cp "$OUT" "${S3_BUCKET}/${TENANT}/" --quiet

    echo "OK: ${TENANT} -> ${OUT}"
done

# Limpar locais antigos (mantém em S3)
find "$BACKUP_DIR" -name "*.tar.gz" -mtime +1 -delete

# Limpar S3 com lifecycle (configurar bucket policy: 30 dias)
```

Permissões e cron:

```
chmod +x /opt/iacv/scripts/backup-config.sh

# Cron: a cada hora cheia
crontab -e
```

```
# Adicionar:
0 * * * * /opt/iacv/scripts/backup-config.sh >> /var/log/iacv-backup.log 2>&1
```

Backup pré-deploy (manual, antes de release/hotfix)

```
ssh prod-host
mkdir -p /backups/predeploy

TS=$(date +%Y%m%d-%H%M)
for TENANT in $(docker stack ls --format '{{.Name}}' | grep '^vision-'); do
  docker run --rm \
  -v ${TENANT}_config:/data:ro \
  -v /backups/predeploy:/backup \
  alpine tar czf /backup/${TENANT}-${TS}.tar.gz /data
done

# Validar integridade dos tarballs
for f in /backups/predeploy/*-${TS}.tar.gz; do
  tar tzf "$f" > /dev/null && echo "OK: $f" || echo "FALHA: $f"
done

# Subir para S3 (segundo destino – defesa em profundidade)
aws s3 sync /backups/predeploy/ s3://iacv-backups/predeploy/${TS}/
```

Não prosseguir com deploy se algum tar tiver falha.

Restore — Um único tenant

Caso um tenant precise voltar a um ponto no tempo:

```
ssh prod-host
TENANT=tenant_x
TS_RESTORE=20260427-1430 # data/hora do backup desejado

# 1. Parar o serviço
docker service scale vision-${TENANT}_vision=0
sleep 10

# 2. Backup do estado atual (caso precise voltar)
docker run --rm \
-v vision-${TENANT}_config:/data:ro \
-v /backups/restore-rollback:/backup \
alpine tar czf /backup/vision-${TENANT}-before-restore-$(date +%Y%m%d-%H%M).tar.gz /data

# 3. Baixar backup do S3 se não estiver local
aws s3 cp
s3://iacv-backups/config-db/vision-${TENANT}/vision-${TENANT}-${TS_RESTORE}.tar.gz \
/tmp/restore.tar.gz

# 4. Limpar volume e restaurar
docker run --rm \
-v vision-${TENANT}_config:/data \
alpine sh -c 'rm -rf /data/*'

docker run --rm \
-v vision-${TENANT}_config:/data \
-v /tmp:/backup:ro \
alpine tar xzf /backup/restore.tar.gz -C /
```

```
# 5. Subir
docker service scale vision-${TENANT}_vision=1

# 6. Validar
sleep 30
curl -fs https://vision.${TENANT}.iacv.example.com/version
docker service logs --tail 50 vision-${TENANT}_vision
```

Restore — Disaster Recovery (host inteiro perdido)

Cenário: VPS de produção perdido. Precisa recriar tudo.

DR.1 — Provisionar novo host

- [] VPS novo com mesma especificação (CPU, RAM, disco)
- [] Instalar Docker, Docker Swarm
- [] Restaurar `/etc/letsencrypt`, configurações Traefik
- [] Apontar DNS (TTL baixo durante DR)

DR.2 — Restaurar volumes a partir do S3

```
# Listar tenants que existiam
aws s3 ls s3://iacv-backups/config-db/ | awk '{print $2}' | sed 's|/||'

# Para cada tenant
TENANTS=$(aws s3 ls s3://iacv-backups/config-db/ | awk '{print $2}' | sed 's|/||')
for TENANT in $TENANTS; do
# Pegar o backup mais recente
LATEST=$(aws s3 ls s3://iacv-backups/config-db/${TENANT}/ | sort | tail -1 | awk '{print $4}')

# Criar volume
docker volume create ${TENANT}_config

# Baixar e restaurar
aws s3 cp s3://iacv-backups/config-db/${TENANT}/${LATEST} /tmp/${TENANT}.tar.gz

docker run --rm \
-v ${TENANT}_config:/data \
-v /tmp:/backup:ro \
alpine tar xzf /backup/${TENANT}.tar.gz -C /

echo "OK: ${TENANT}"
done
```

DR.3 — Recordings (apenas do S3)

Os recordings ficam **somente no S3** (não em backup local — ocupam TB). A app, ao subir, consulta o S3 para apresentar histórico.

Validar que o `frigate/s3_sync.py` está apontando para o bucket correto via env vars:

```
ICV_S3_ENDPOINT
ICV_S3_BUCKET
ICV_S3_ACCESS_KEY
ICV_S3_SECRET_KEY
```

DR.4 — Subir os stacks

```
# Para cada tenant, deploy do stack a partir do template
for TENANT in $TENANTS; do
ICV_TENANT_SLUG=$TENANT \
ICV_LICENSE_KEY=$(get_license_for $TENANT) \
docker stack deploy -c docker-compose.cloud.yml vision-${TENANT}
done
```

DR.5 — Validação

```
for TENANT in $TENANTS; do
echo -n "${TENANT}: "
curl -fs --max-time 10 https://vision.${TENANT}.iacv.example.com/version || echo "FALHOU"
done
```

Teste de DR (recomendado: trimestral)

Para garantir que o restore **realmente funciona**:

- Provisionar VPS de teste (descartável)
- Executar DR.1 a DR.4 com tenants reais (ou amostra)
- Validar que dados aparecem corretamente
- Documentar tempo total (atualizar RTO se necessário)
- Destruir o VPS de teste

Sem teste, o backup é só esperança — não é seguro.

Política de retenção (resumo)

```
config.db:
- Local (host de prod): últimos 7 dias (24/dia = 168 versões)
- S3 (versionado): 30 dias rolling
- Pré-deploy: mantido por 90 dias (segurança extra)

Recordings:
- S3: 90 dias (lifecycle policy)
- Glacier (long-term): 1 ano (eventos com label "evidence")

Snapshots:
- S3: 30 dias

Logs (Loki):
- Ativos: 14 dias
- S3 archive: 1 ano
```

Comandos rápidos (cola)

```
# Backup imediato de 1 tenant
TENANT=xxx
docker run --rm -v vision-${TENANT}_config:/data:ro \
-v /backups/manual:/backup alpine \
tar czf /backup/${TENANT}-${date +%Y%m%d-%H%M}.tar.gz /data
```

```
# Listar backups disponíveis (S3)
aws s3 ls s3://iacv-backups/config-db/vision-${TENANT}/ --human-readable

# Validar último backup (extrair e checar)
aws s3 cp s3://iacv-backups/config-db/vision-${TENANT}/${aws s3 ls ... | sort | tail -1 |
awk '{print $4}' /tmp/check.tar.gz
tar tzf /tmp/check.tar.gz | head
```


Anexo A. Comandos Uteis

A.1 Status geral

```
# Versoes em execucao por tenant
for s in $(docker service ls --format '{{.Name}}' | grep '^vision-'); do
echo -n "$s: "
docker service inspect "$s" \
--format '{{.Spec.TaskTemplate.ContainerSpec.Image}}'
done

# Versao por endpoint /version
for t in tenant_a tenant_b; do
echo -n "$t: "
curl -s https://vision.$t.iacv.com/version
done
```

A.2 Logs

```
# Tail dos ultimos 5 min de um tenant
docker service logs -f --since 5m vision-{{TENANT}}_vision

# Logs de todos os tenants (centralizado em Loki)
# (no Grafana) {service_name=~"vision-.*"} |~ "ERROR"
```

A.3 Manutencao

```
# Espaco de cada volume
docker system df -v | grep vision-

# Limpar imagens antigas
docker image prune -a --filter "until=168h"

# Forcar redeploy (rebaixar tag)
docker service update --force vision-{{TENANT}}_vision
```

Anexo B. Template de Postmortem

Toda incidencia P0 ou P1 tem postmortem em 48 horas. Use este template:

```
# Postmortem - INC-YYYY-MM-DD-001

## Resumo
<2-3 frases descrevendo o que aconteceu>

## Severidade: P0 | P1
## Duracao: HH:MM - HH:MM (XX minutos)
## Tenants impactados: <lista>

## Timeline (UTC-3)
- 14:32 - Deploy v1.2.3 iniciado em todos os tenants
- 14:35 - Alerta de erro 5xx em tenant_a
- 14:36 - On-call notificado
- 14:38 - Confirmado impacto, iniciado rollback
- 14:42 - Rollback completo, validado /version

## Root cause
<O que causou? Use 5 whys>

## Impacto
- X tenants offline por Y minutos
- Z gravacoes nao realizadas
- Reclamacoes recebidas: N

## O que funcionou
- Alerta disparou em < 1 min
- Rollback foi rapido (3 min)

## O que falhou
- Smoke test nao cobria cenario X
- Logging nao mostrou Y claramente

## Acoes preventivas (cada uma vira issue)
- [ ] Adicionar smoke test cobrindo X (issue #123)
- [ ] Melhorar log de Y (issue #124)
- [ ] Atualizar runbook 02 (issue #125)

## Lessons learned
<O que voce ensinaria a outro engenheiro>
```

Fim do documento.

Este plano e versionado junto com o codigo em
docs/producao/IACV-Plano-Producao.pdf